Vehicle Routing Competition

EURO Meets NeurIPS 2022

[Team4sake]

Chanon A., Duangporn K., Malin H., Palita T., Tharapoom H.

Vehicle Routing Problem (VRP)

VRP is a generalization of the traveling salesman problem, which traditionally has been studied in the field of Operations Research (OR) and solutions are proposed in space of feasible solution.

With the influence of Machine Learning in these recent years, many ML researchers aim to train model, exploit patterns to solve new problem instance with better and faster solution.

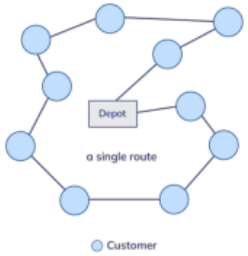
reference picture

Vehicle Routing Competition

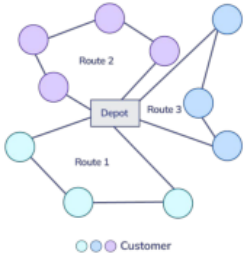
Goal for this competition is to solve the vehicle routing problem with time windows in two different settings, **static** and **dynamic**.

For static setting, it is the capacitated vehicle routing problem where customer requires delivery within a specific interval of time or time window. And for dynamic setting, new customer request may arrive at different epoch during the day.

Static



Dynamic



reference picture

Baseline Strategy

For static baseline:

- We have Hybrid Genetic Search algorithm (HGS) as the current state-of-the-art.

For dynamic baseline:

Greedy	every available request is going to be dispatched within the current epoch.
Lazy	only must-go requests will be dispatched.
Random	requests to be dispatched in the current epoch are must-go and randomly selected requests: each request is dispatched with 50 % probability, independently.

Strategies Proposed

1. **Cutoff**
   dispatch nodes within a fixed radius (50^{th} percentile of average distance) around must-go customers
2. **Modified cutoff (modcutoff)**
   same idea, but the cutoff radius grows with each epoch (70^{th} percentile $+ 8\,\%$ per epoch)
3. **Centroid modified cutoff (CMC)**
   measures the cutoff radius from the actual centroid of the cluster instead of the must-go node
4. **Ensemble**
   logistic regression picks modcutoff or CMC per instance
5. **Logistic**
   postpones a single customer at the start epoch, then follows greedy

Cutoff

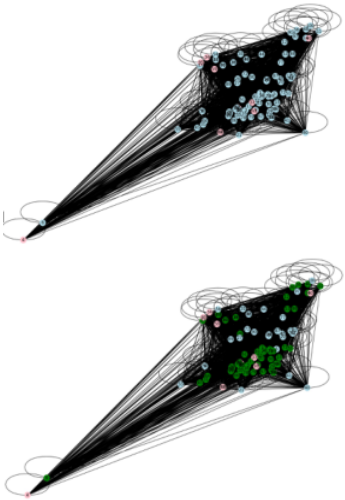
Background: From network visualization, we believe that we should dispatch nearby must-go customers.

Key idea: To find nearby must-go customers to dispatch

- **Set cutoff threshold:** 50^{th} percentile of average distance in this instance (threshold varies instance to instance)
- **Dispatch nodes within cutoff radius of must-go node**

note that: we have

pink node as must-go customers
blue node as to-postpone customers
green node as dispatched customers

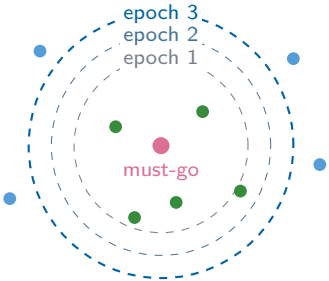


Modified Cutoff (modcutoff)

Background: From cutoff strategy, we believe that the greater the number of epochs, the smaller the number of nodes to be delayed.

Key idea: To find nearby must-go customers to dispatch, varying the number of nodes to be dispatched according to the number of epochs

- **Set cutoff threshold:** 70^{th} percentile of average distance in this instance in the first epoch then increase by additional 8 percent in each additional epoch
- **Dispatch nodes within cutoff radius of must-go node**



cutoff radius +8 % per epoch:

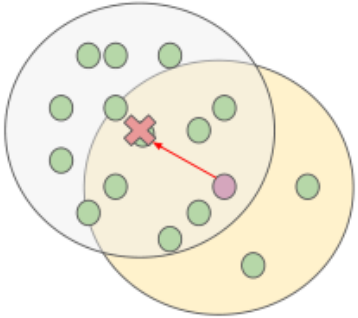
more nodes dispatched, fewer delayed

Centroid Modified Cutoff (CMC)

Background: From cutoff strategy, the centroid used to measure the distance should be the actual center of a cluster, so more nearby nodes in cluster are dispatched compared to Modcutoff.

Key idea: To find nearby centroid customers to dispatch

- **Set cutoff threshold:** 70^{th} percentile of average distance in this instance in the first epoch then increase by additional 8 percent in each additional epoch
- **Dispatch nodes within cutoff radius of actual centroid node**



Ensemble

Background: From performance of **modcutoff** and **CMC** strategy, we notice that one strategy outperforms another strategy in each instance. So, we should select a suitable strategy for each instance.

Key idea: To select strategy which is suitable for each instance

- **Collect features and cost from environment of 250 instances**

  x_1 = minimum number of customers within radius 25^{th} percentile of distance
  x_2 = maximum number of customers within radius 25^{th} percentile of distance
  x_3 = 25^{th} percentile number of customers within radius 25^{th} percentile of distance
  x_4 = 50^{th} percentile number of customers within radius 25^{th} percentile of distance
  x_5 = 75^{th} percentile number of customers within radius 25^{th} percentile of distance
  Y = 1 if cost of CMC is better than cost of modcutoff, otherwise 0

- **Build logistic regression model to predict likelihood of CMC being better than modcutoff**

- **Choose a particular strategy to use for instance**

Logistic

Background: Since we have greedy (without postponing) as the best baseline, we believe that we should not postpone many customers at the same time. So, we suggest postponing only one at the start epoch.

Key idea: To find only one customer to postpone at the start epoch and use greedy strategy for other epochs

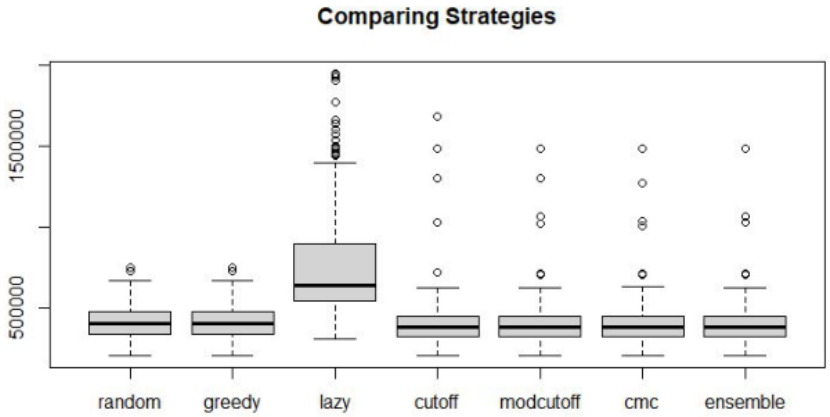
- **Collect features and cost from environment of 250 instances**

  x_1 = number of customers within radius 25^{th} percentile of distance
  x_2 = number of customers between radius 25^{th} and 50^{th} percentile of distance
  x_3 = number of customers between radius 50^{th} and 75^{th} percentile of distance
  x_4 = demand per capacity
  x_5 = sum of average distance within radius 25^{th} percentile
  x_6 = sum of average distance within radius 50^{th} percentile
  x_7 = sum of average distance within radius 75^{th} percentile
  x_8 = sum of average distance within mean radius

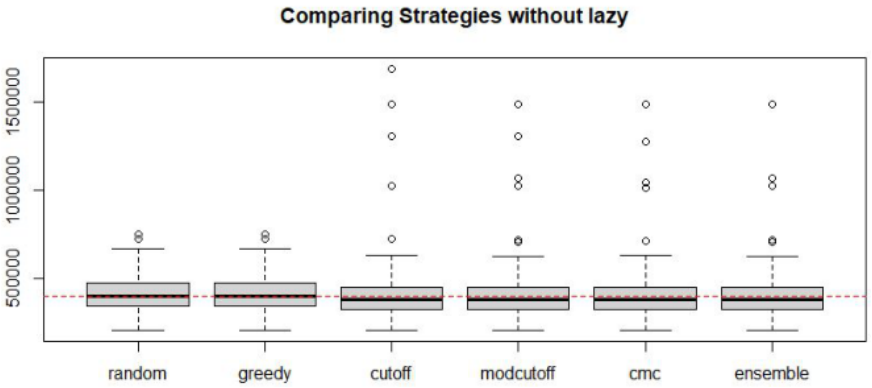
  $Y = 1$ if cost of push_to_go is better than cost of greedy, otherwise 0

- **Build logistic regression model to predict likelihood of being better than greedy**
- **Choose one customer who has the most likelihood and postpone at the start epoch**

Comparing Strategies (1)



Comparing Strategies (2)



Leaderboard (qualification)

The top 10 on this leaderboard at October 31st will be evaluated on the final dataset. Each submission gets two ranks: one based on static performance and one based on dynamic performance. The submissions are ranked by the average of these two ranks, with the average cost as tie-breaker.

Rank	Date	Team	Static cost	Dynamic cost	Avg. cost	Static Rank	Dynamic Rank	Avg. rank
1	10/20/22	Kléopatra	180609.7	335405.2	2.580075e+05	4.0	1.0	2.5
2	10/19/22	HowToRoute	180570.3	349115.4	2.648428e+05	1.0	5.0	3.0
3	10/29/22	ORberto Hood and the	180677.0	346094.9	2.633860e+05	6.0	4.0	5.0
24	10/27/22	LAZY SOTA	180842.6	368400.6	2.746216e+05	28.0	26.0	27.0
25	10/05/22	hq	180852.9	354411.9	2.676324e+05	38.0	19.0	28.5
26	10/25/22	team4sake	180843.0	360945.4	2.708942e+05	33.0	24.0	28.5
27	10/29/22	CH_JointTeam	181564.4	352182.4	2.668734e+05	49.0	9.0	29.0